Sarita Sangvez
23i - 2088
CY - A.

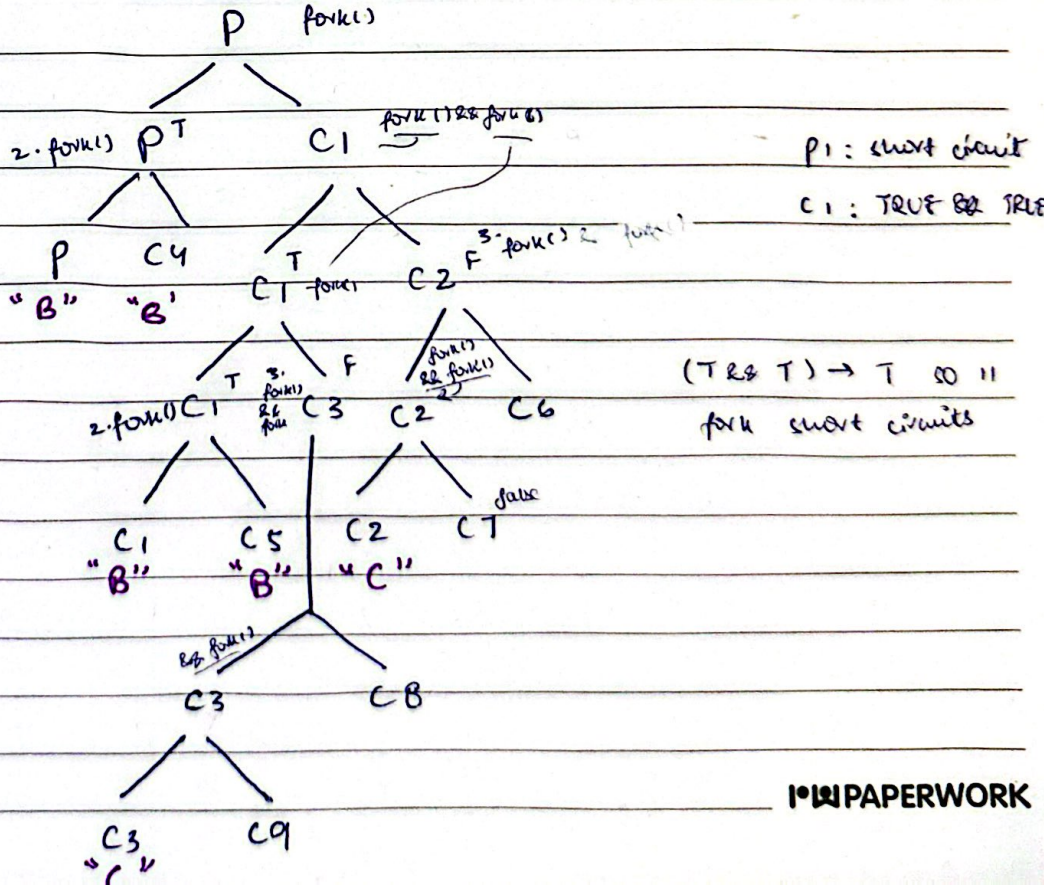
Operating Systems.
Assignment 02

Q 1.

0. "A" printed once
1. if (fork() || (fork() && fork())) {
2. fork(); printf("B")
} else {
3. if ((fork() && fork()) || fork())
{ printf("C") }
}
}
4. printf("D");
everyone executes "D"

Output : A B B B B C C D D D D D D D D D D

1. Tree :



2. Every process prints D as the printf("D") statement is outside all if-else blocks and execution will reach the end of program before exiting. (10 times)

3. "B" is printed by P, C1, C4, C5
"C" is printed by C2, C3.

OS Assignment 02

Q2. 1. If f1 executes first:

1. x = 1, T1 sleeps so T2 runs.
   doesn't go in if (x==0)
   prints ("T2").
   x becomes 2.
   T1 wakes up
   prints T1.
   y becomes 2.

Possible outputs? "T2 T1 Main" OR "T1 T2 Main" as it depends on when T1 comes back from sleep.

2. If f2 executes first
   x = 0 so y = 1
   prints "T2"
   x = 2
   T1 then executes
   x = 1
   y = 1 so no sleep
   y = 2.

Possible outputs: T2 T1 Main

2. If f1 runs first, then y is 0 and T1 sleeps which means there's more chance of T2 to execute first. So, sleep affects the race outcome.

3. we can move sleep(1) to f2 function. That way, T1 will definitely print first.
   Output: T1 T2 Main.

Q3.

1. Main creates thread T0.
   id = 0 < 3 , T0 creates T1
   id = 1 < 3 , T1 creates T2
   id = 2 < 3 , T2 creates T3
   id = 3 < 3 ✗
   So, Main → T0 → T1 → T2 → T3.

2. Total threads (w/o main) = 4.

3. Every thread joins its child so the parent waits/blocks for its child to finish.
   e.g T0 is alive and creates T1 and then waits. So both are alive.
   Max threads alive at once = 2.

4. Enter-0 Enter-1 Enter-2 Enter-3 Exit-3 Exit-2 Exit-1 Exit-0 Done.
   (LIFO, deepest-first)

5. Parent does not wait for the child in detach()
   All "Enter" statements will print first but the "Exit" statements will be random and can not be predicted.
   e.g Enter-0 Enter-1 Enter-2 Enter-3 Exit-1 Exit-0 Exit-3 Exit-2. Done

Q4. step = 0, so only A runs (does not go in while) prints "A" and step = 1.
pthread_cond_signal → wakes up the next thread
pthread_mutex_unlock → A finishes and exits
Now only B can run as step = 1.
Then only C runs as step = 2

1. output : A B C Main

2. If we remove wait condition in B, it may run before A sets step to 1. The output may change to :
B A C Main or B C A Main.
pthread_cond does not guarantee order of wake up so we use while.

3. We can use a single shared variable eg step = 0.
All threads will continuously check the value of step in a while loop (busy waiting).
Thread only executes when step matches condition
After executing, thread will update step for the next thread to run.

Q5.

1. Both Threads will run 3 times each and take turns as we use mutex.
Possible interleavings:

T1 : 1 2 3 OR T1 : 1
T2 : 4 5 6 T2 : 2
T : 3 Final : 6
Final : 6. T : 4
T : 5
T : 6

2. without mutex, both threads will be able to modify the variable so it may get overwritten by one or the other.

e.g T_1 reads counter = 0; T_2 also reads counter = 0,
T_1 writes counter = 1, T_2 also writes counter = 1
Ans may become: 1 1 2 2 3 3

3. Interleavings: All different possible ways the CPU can interleave the execution of instructions from multiple threads. The final output is still correct with interleaving as mutex only allows one thread to access the counter at a time. So, the part mutex_lock() ..... mutex_unlock() works like a single operation. Even if the print order is mixed, final output is always correct.